

Git Cheatsheet

🚧 开始工作前，请务必先建分支！`git switch -c <新分支名>`

设置

设置附加到提交和标签的姓名和邮箱

```
git config --global user.name "你的姓名"
git config --global user.email "name@email.com"
```

配置 Git 代理 (HTTP/HTTPS)

```
git config --global http.proxy http://127.0.0.1:7890
git config --global https.proxy http://127.0.0.1:7890
```

强制使用 LF 换行符

```
git config --global core.autocrlf input
```

开始项目

创建本地仓库 (省略目录初始化当前目录)

```
git init <目录>
```

下载远程仓库

```
git clone <仓库地址>
```

忽略文件 (.gitignore)

常见忽略规则示例

```
*.log # 忽略所有 .log 文件
node_modules/ # 忽略目录
.env # 忽略环境配置
!main.log # 例外: 追踪 main.log
```

进行修改

添加文件到暂存区

```
git add <文件>
```

暂存所有文件

```
git add .
```

提交所有暂存的文件

```
git commit -m "提交信息"
```

修改最后一次提交 (修正信息或补漏)

```
git commit --amend -m "新的提交信息"
```

分支

列出分支 (-r 远程, -a 所有)

```
git branch
```

查看分支及跟踪的远程分支

```
git branch -vv
```

为当前分支关联远程分支

```
git branch -u <别名> /<远程分支>
```

为指定分支关联远程分支

```
git branch -u <别名> /<远程分支> <本地分支>
```

创建新分支

```
git branch <新分支名>
```

切换分支

```
git checkout <分支名>
```

```
git switch <分支名>
```

切换到远程分支

基于远程分支创建并跟踪同名分支、同时切换过去进行开发

```
git checkout -t <别名> /<分支>
```

```
git switch -t <别名> /<分支>
```

创建并切换分支

git switch 是新版命令，语义更清晰

```
git checkout -b <新分支名>
```

```
git switch -c <新分支名>
```

删除分支 (-d 已合并, -D 强制)

```
git branch -d <分支名>
```

```
git branch -D <分支名>
```

打标签

```
git tag <名>
```

```
git tag -a <名> -m "msg"
```

查看标签

```
git tag
```

```
git show <标签名>
```

合并

合并分支 a 到 b (--no-ff 非快速)

```
git checkout b
```

```
git merge a
```

放弃合并 (冲突后)

```
git merge --abort
```

压缩合并 (所有提交压缩为一个)

```
git merge --squash a
```

摘樱桃 (应用特定提交)

```
git cherry-pick <commit>
```

摘樱桃冲突处理

```
git cherry-pick --continue
```

```
git cherry-pick --abort
```

变基 (Rebase)

将特性分支变基到主分支

保持历史整洁，防止不必要的合并提交

```
git checkout feature
```

```
git rebase main
```

变基冲突处理

```
git rebase --continue
```

```
git rebase --abort
```

交互式变基 (清理提交)

```
git rebase -i main
```

交互式最后3个提交

```
git rebase -i HEAD~3
```

Todo 列表常用命令

pick: 保留

reword: 改提交信息

edit: 暂停修改内容

squash: 合并到上一条

fixup: 合并但丢弃信息

drop: 丢弃

撤销操作

移动/重命名并暂存

```
git mv <原> <新>
```

删除文件并暂存

```
git rm <文件>
```

仅从暂存区删除 (保留文件)

```
git rm --cached <文件>
```

查看/临时检出提交 (Detached HEAD)

```
git checkout <commit-ish>
```

```
git switch --detach <...>
```

创建撤销提交 (反向操作)

```
git revert <commit-ish>
```

丢弃工作区修改

```
git checkout -- <文件>
```

```
git restore <文件>
```

取消暂存 (保留工作区)

```
git restore --staged <文件>
```

Reset 重置分支指针

```
git reset --soft <commit>
```

移动 HEAD，保留暂存/工作区

```
git reset <commit>
```

重置暂存区，保留工作区

```
git reset --hard <commit>
```

🚧 重置暂存和工作区

清除未跟踪文件 (-n 预览)

```
git clean -nfd
```

```
git clean -fd
```

查看仓库

查看状态 (s: 紧凑)

```
git status
```

```
git status -sb
```

查看历史

```
git log --oneline
```

```
git log --graph --oneline --decorate
```

```
--all
```

查看对象详细信息

```
git show <commit-ish>
```

查看差异

```
git diff # 工作区 vs 暂存
```

```
git diff --cached # 暂存 vs HEAD
```

```
git diff c1 c2 # 提交间差异
```

引用日志 (找回丢失提交)

```
git reflog
```

查看每行修改人 (Blame)

```
git blame <文件>
```

储藏 (Stash)

暂存更改 (-u 含未跟踪)

```
git stash
```

```
git stash push -m "msg"
```

列出所有储藏

```
git stash list
```

应用最近储藏 (保持堆栈)

```
git stash apply
```

弹出储藏 (从堆栈删除)

```
git stash pop stash@{2}
```

应用并恢复暂存区状态

```
git stash pop --index
```

删除储藏

```
git stash drop stash@{0}
```

```
git stash clear
```

同步

管理远程仓库

```
git remote add <别名> <url>
```

```
git remote -v
```

获取分支 (不合并)

```
git fetch <别名>
```

```
git fetch --prune <别名>
```

获取并合并

```
git pull
```

```
git pull --rebase <别名>
```

推送

```
git push <别名>
```

推送到指定分支

```
git push <别名> <分支>
```

推送本地到不同名远程分支

```
git push <别名> <本>:<远>
```

首次推送并设置追踪

```
git push -u <别名> <分支>
```

推送标签

```
git push <别名> --tags
```

强制本地分支同步远程分支

放弃未提交的改动，用于处理本地混乱的提交历史

```
git reset --hard <别名> /<分支>
```

压缩合并 (Squash Merge)

合并变更但不生成提交，将目标分支的多次提交“压缩”成一次未提交的修改

```
git merge --squash <分支>
```

强制推送 (慎用)

覆盖远程分支历史，仅用于自己独占的分支

```
git push --force <别名> <分支>
```

工作流: Fork 模式开发

核心定义

[分支] main: 镜像(只同步) | dev: 纯净(提PR) | local-dev: 本地(含配置)

[仓库] upstream: 官方 | origin: 个人 Fork

1. 同步官方代码 (Daily Routine)

```
git switch main
git fetch upstream
git reset --hard upstream/main # 强制一致
git push origin main --force
```

2. 日常开发 (在 local-dev)

```
git switch local-dev
git rebase main # 同步更新
# ... 正常开发 ...
git add . && git commit -m "WIP..."
```

3. 提交流程 (清洗代码 -> Dev)

```
git switch dev
git reset --hard main # 重置基准
git merge --squash local-dev # 压缩合并
# ... 剔除配置 ...
git commit -m "Feat: ..."
git push origin dev -f
```

基本概念

main / master

默认开发分支

origin

默认远程仓库别名

HEAD

指向当前分支的指针

HEAD^ / HEAD~4

父提交 / 第4代祖先

什么是 commit-ish?

任何能被解析为提交的引用:

- 哈希: a1b2c3

- 分支/标签: main, v1.0

- 相对引用: HEAD~